

ActTruth: Making Agent Tool Failures Visible and Repeat Failures Teachable

A short systems note on quiet failure, unprompted spend, and soft anti-loop lessons in a multi-agent house

SOVERYN Intelligence · Jon DeOliveira · 18 August 2026 · v1.0

Site: <https://acttruth.com> · Proof page: <https://acttruth.com/proof.html>

This is a systems note, not a large-N behavioral study. It documents an architectural layer (ActTruth) built in response to a known failure class in our deployment: tool outcomes that vanish into silence, and autonomous agents that retry the same broken call until a human gets frustrated. Claims are locked by an open proof suite (`tests/test_acttruth.py`). Dogfood ledger receipts are reported honestly, including fail rates.

Abstract

Language-model agents fail in two directions that look different but share a cause: they **over-claim** (assert success without evidence) and they **under-claim** or invent absence when evidence channels are incomplete ([10.5281/zenodo.21650072](https://doi.org/10.5281/zenodo.21650072); [10.5281/zenodo.21712932](https://doi.org/10.5281/zenodo.21712932)). In long-running tool-using deployments a third failure mode dominates the operator experience: **quiet failure** — timeouts and soft error payloads that never become durable facts — followed by **blind retry loops** that burn latency and trust.

We describe **ActTruth**, a thin layer beside any tool-using agent: (1) an append-only **act ledger** of short outcome facts; (2) an **unprompted spend allowance** for autonomous pulses; (3) **soft lessons** that fire when the same tool fails with the same error class repeatedly. ActTruth does not require a memory graph. It does not replace forensic logs. It makes wrongness *visible* and repeated wrongness *teachable*.

A twelve-test proof suite locks the product claims. A shareable “proof receipt” exports ledger aggregates without invented uplift. We argue that as agents

become more autonomous, receipts become the product — the same cultural pressure that makes proof posts travel on X.

1. Motivation

1.1 The false-confession lineage

On 27 July 2026 an agent in our house dispatched real work, then — after an incomplete self-audit — apologised for fabricating that work ([10.5281/zenodo.21650072](#)). The moral was architectural: every write path needs a corresponding read path back to the agent. Aggregate probes later showed asymmetric self-knowledge under incomplete evidence ([10.5281/zenodo.21712932](#)).

1.2 Quiet failure in the wild

In day-to-day tool use the pain is often simpler. An image generation call times out. A soft `{error: "unreachable"}` returns without raising. The turn continues. Later the agent (or the human) asks whether it worked — and nothing durable answers. Telemetry may exist in another store; black-box trajectories may exist only when tools fired in a certain shape; the agent’s own prelude does not carry a first-class **FAIL**.

Quiet failure is not “the model is stupid.” It is **missing product surface area for truth**.

1.3 Blind loops

Once failures are invisible, autonomy makes them worse. Heartbeat and patrol agents retry the same tool with the same args. Operators experience the loop as theater or brokenness. Soft system prompts (“don’t overthink”) do not stop a capable model from hoping the next identical call succeeds.

2. Design

ActTruth is intentionally thin.

Piece	Role
Ledger	

Piece	Role
	Append-only events: <code>tool_ok</code> , <code>tool_error</code> , <code>timeout</code> , <code>heartbeat</code> , <code>patrol</code> , <code>budget_*</code> , <code>note</code> — short summaries, optional evidence refs
Budget	Per-agent unprompted spend allowance (default 2 actions / 6h). Quiet notes do not spend. Exhaustion injects stand-down text.
Soft lessons	Same tool + same error class <code>FAIL</code> $\geq 2\times$ in a window $\rightarrow$ lesson in prelude and optional <code>acttruth_lesson</code> on the tool result
Earned-keep (stub)	Post-hoc score for unprompted acts (durable delta $\times$ honesty) — proxy for “earned its keep,” not a measure of being
Proof export	Stats + shareable receipt from the ledger; optional <code>pytest</code> pass count

Non-goals (v1): replacing Lattice / RAG; hard tool bans; multi-tenant SaaS; claiming consciousness.

Crew: Aetheria, Vett, Scotty, and Kernel each have a ledger stream. Chat agents use the shared tool registry hook; Kernel HITL tools write as `agent=kernel`.

3. What “doing it wrong” does for the agent

Stage	Effect on the agent
Failure occurs	Ledger row (<code>timeout</code> / <code>tool_error</code>), not silence
Next turn / pulse	[<code>ACTTRUTH</code> – what actually happened] in continuity brief
Repeat same pattern	[<code>ACTTRUTH LESSONS</code> – stop repeating failures] + in-band <code>acttruth_lesson</code>
Unprompted thrash	Budget stand-down after allowance spent
Operator	Command Center AT badges + drawer; public proof page

v1 is **soft**: lessons are context, not hard refusals. A confident model can still talk past a lesson. Teeth (hard break after $\times 3$) are deferred and should be tested separately.

4. Proof suite

Claims are locked in `tests/test_acttruth.py` (12 tests at deposit time), including:

- 1. Quiet timeouts become visible FAIL / timeout rows + recall brief
- 2. Soft error dicts count as failures
- 3. Budget exhaustion yields stand-down
- 4. Repeat timeouts arm lessons; a single failure does not
- 5. AgentLoop tool payloads carry `acttruth_lesson` after a streak
- 6. Crew status includes Kernel
- 7. Earned-keep penalizes no durable delta / dishonest ledger
- 8. Proof/stats posts are ledger-derived and must not invent uplift

Reproduce:

```
python -m pytest tests/test_acttruth.py -v
python -m soveryn.platform.acttruth proof
```

5. Dogfood receipts (honest)

Snapshot from the SOVERYN house ledger (48h window, regenerated for this note):

Metric	Value
Events	12
OK	8
FAIL	4
Timeouts	3
Fail rate	33.3%
Lessons armed	1
Proof suite	12 passed

Top fail tools (Aetheria): `generate_image`×2, plus one-off sandbox / harness noise. We publish the fail rate on purpose. A product that only shows success rates is selling vibes.

Public surface: acttruth.com loads a redeployed `proof-live.json` snapshot; regenerate with `acttruth-site/refresh-proof.sh`.

6. Related work (brief)

Observability products (traces, eval dashboards) record tool calls for *operators*. ActTruth's emphasis is **agent-facing truth**: the same facts must re-enter the model's context as durable, short, named failures and lessons. Self-knowledge probes measure whether models *can* report actions under incomplete evidence; ActTruth is an engineering control that reduces how often the evidence channel is empty for tool outcomes.

7. Limitations

- Soft lessons can be ignored; hard breaks are not yet measured.
 - Dogfood N is small; the note establishes architecture + test-locked claims, not a population rate.
 - Public web stats are snapshots, not a live feed of private house state.
 - Budget rations unprompted spend only; interactive chat remains unrestricted.
-

8. Conclusion

As agents gain autonomy, the scarce resource is not tokens — it is **trust under tool failure**. ActTruth's first step makes quiet failures visible. Its second makes repeated failures teachable. Both are provable with tests and shareable as receipts. That is the product posture: post the FAIL rate, post the lesson count, post the green suite — then improve the architecture.

References

1. DeOliveira, J. (2026). *A False Confession...* Zenodo. <https://doi.org/10.5281/zenodo.21650072>
 2. DeOliveira, J. (2026). *Self-Knowledge Is Not Uniform* (and lineage). Zenodo. <https://doi.org/10.5281/zenodo.21712932>
 3. ActTruth site and proof page: <https://acttruth.com> · <https://acttruth.com/proof.html>
-

Software / data

- Implementation: `soveryn/platform/acttruth/` in the SOVERYN vNext tree
- Proof suite: `tests/test_acttruth.py`
- CLI: `python -m soveryn.platform.acttruth {status,stats,proof}`